🧠 Why I Chose a Local AI-Agent + Cloud-Model Architecture for My Large Solo Development Project

1. Amar project, amar workflow, ebong ami asole ki solve korte chacchi

Ami ekta khub boro software project solo develop korchi:

Project: GarTexHub — A Person's Website / A-Peronson-s-Website

Ei project-ta just ekta simple website na. Public repository-r README onujayi, eta ekta large B2B textile and garments marketplace, jekhane React frontend, Express backend, database, authentication, real-time communication, search, AI integration, admin panel, governance, testing, ebong deployment-related onek complex system ache.

README-te React 19 SPA, Express 5 API, Prisma/PostgreSQL, WebSocket/SSE, multiple AI-provider integration, background workers, authentication, testing, ebong onek modular service-er architecture describe kora ache.

Project reference: https://github.com/gamertoky1188gro/A-Peronson-s-Website/

Ei scale-er project solo develop korar shobcheye boro challenge holo sudhu code lekha na. Ekshathe onek kichu manage korte hoy:

- Existing codebase bujha ebong relevant files identify kora.
- Frontend, backend, database, API, authentication ebong shared logic-er dependency maintain kora.
- Alada feature-er implementation parallelize kora.
- Build errors, regression, runtime problems ebong integration conflicts investigate kora.
- Testing, reviewing, documentation ebong deployment-related kaj manage kora.
- Fast development-er sathe sathe code quality o architectural consistency protect kora.

Ami ei challenge handle korar jonno ekta carefully managed AI-agent workflow use korchilam.

Amar strategy chilo **16+ specialized sub-agent-ke task onujayi organize kora**, prottekta task-er jonno suitable model choose kora, ebong jekhane possible free AI usage diye maximum development output

ber kore ana.

Important distinction holo, 16+ agent use korar mane ei na je prottek agent-ke highest-end model diye chalate hobe. Agent-er responsibility, task complexity, available model capability, context requirement, rate limit ebong verification cost consider kore model select kora uchit.

Amar goal chilo:

Maximum verified development output per free AI request — not maximum intelligence consumption per task.

Ejonnoi ami model choice, agent delegation, parallel work, context management ebong fallback options niye carefully plan korechilam.

---

2. Tahole OpenCode amar workflow-er jonno important chilo keno?

OpenCode amar kache important chilo karon eta sudhu question-answer chatbot na. Eta amar existing development environment-er moddhe theke coding agent hisebe kaj korte pare.

Official documentation: https://opencode.ai/docs/

Amar workflow-e OpenCode-er main advantages chilo:

1. Existing local codebase-er access

Project files repeatedly upload kore explain korar bodole agent local repository inspect korte, relevant files porte, changes korte ebong terminal-e build/test run korte parto.

2. Model selection-er flexibility

Ami available models-er moddhe task onujayi model choose korte partam. Amar existing setup-e normally 5–7 ta usable model option thakto bole ami ekta model unavailable hole onnotar dike jete partam.

Eta important chilo, karon amar sob task-er jonno ekoi reasoning capacity dorkar chilo na.

3. Specialist agent workflow

Different agents-ke alada responsibilities deya jeto — repository exploration, feature implementation, bug investigation, testing, code review, architecture analysis ebong integration support.

4. Existing developer tools-er sathe integration

Terminal, Git, project files, build output, test results ebong local development setup-er sathe agent-er kaj directly connected thakto.

5. Free-first development

Amar workflow-er ekta central condition chilo paid AI subscription ba paid API usage avoid kora. Amar available free models-er moddhe theke choose korar flexibility tai important chilo.

Ami OpenCode choose korechilam ei practical advantages-er combination-er jonno. Eta kono permanent guarantee chilo na je free usage kokhono rate-limited hobe na.

Kintu ekhon problem-ta kothay?

Got 2–3 din dhore OpenCode free-model workflow-e emon rate-limit issue hocche je model use na kore wait korleo, displayed reset time par hoye geleo, notun request dile abar limit error ashe.

Ami OpenCode restart/retry ebong different model try korar moto basic possibilities consider korechi. GitHub-e similar report-o peyechi.

Amar experience-er sathe mile emon current reports:

- OpenCode #59387 — Free-tier issues and reset problems
- OpenCode #59599 — Rate-limit persistence after reset and clean reinstall
- OpenCode #55136 — "Rate limit exceeded. Please try again later."

October 26–27, 2026-er public discussion-e multiple user free models-er quota reset-er por-o restriction thakar report diyechen. OpenCode-er response-o explain kore je free-tier limit ebong reset OpenCode-er hosted service-side-e track hoy; local CLI reinstall korlei server-side quota automatically reset hoy na.

Ei reports amar observed problem-er sathe strongly consistent, kintu egulo dekhe prottek user-er problem-er exact root cause eki bole confirm kora jay na. Eta shared service-side quota issue, accounting/reset bug, account-specific restriction, ba onno relevant service problem hote pare.

Amar main concern holo: ami amar project architecture emon ekta free service-er upor fully dependent rakhte chai na, jar available quota ba reset behavior amar control-e nei.

Ekhane amar approach-ke abar consider kora mane normal chatbot-e switch kora na. Borong local agent architecture rekhe reliable model access, fallback ebong task-based routing improve kora.

---

3. Four types of AI system: konta asole ki kore?

Ekhane char ta architecture alada kore bujha important.

A. Normal AI chatbot

Model cloud-e, interaction mainly chat-based.

Ami prompt dei, AI answer ba code dey. Tarpor amake nijer project-e file identify, code apply, terminal command run, error inspect, test execute ebong next step manage korte hoy — unless chatbot-er sathe additional tools or repository integration configure kora thake.

Advantages:

- Use kora easy.
- Setup minimum.
- Explanation, architecture discussion, brainstorming ebong code review-er jonno useful.
- Local machine-e agent runtime chalate hoy na.

Disadvantages for my project:

- Manual back-and-forth onek bere jay.
- Large codebase-er relevant state repeatedly explain korte hote pare.
- Code generate korlei actual project-e successfully integrate hoyeche, eta guarantee hoy na.
- Build/test feedback abar AI-ke dite hoy.
- 16+ specialist-agent-er coordinated workflow recreate korte alada infrastructure lagte pare.

Normal chatbot diye amar kaj kora impossible na. Kintu amar already-existing local development workflow-er khetre eta onek manual work abar amar upor niye ashbe.

B. Fully cloud-based AI agent

Model ebong agent-er working environment remote infrastructure-e chole.

Agent task plan korte, remote workspace-e files inspect korte, code edit korte, terminal command run korte ebong tests execute korte pare — platform-er capability onujayi.

Advantages:

- Local machine-er resources comparatively kom lage.
- Remote workspace-e long-running task execute kora possible.
- Hosted infrastructure-er compute use kora jay.
- Kichu service task-completion-oriented workflow provide kore.

Disadvantages for my project:

- Remote workspace automatically amar existing local development environment-er equivalent hoy na.

- Local files, installed tools, local services, environment variables ebong ongoing changes access korar jonno repository sync, upload ba bridge lagte pare.
- Remote test pass korleo amar actual local configuration-e same result hobe, eta always guaranteed na.
- Free cloud-agent allowance, execution time, task quota, concurrency ba session limits thakte pare.
- Local code/context remote environment-e transfer korar privacy implications ache.

Cloud agent bhalo option, especially jokhon task remote workspace-e complete kora jay ebong amar PC off thakleo kaj cholte thaka dorkar.

Kintu amar existing local project-e directly kaj kora, local terminal feedback neya, ebong current environment-e changes verify korar jonno eta automatically best option na.

C. Local AI agent + cloud AI model

Agent amar computer-e chole; AI reasoning online model theke ashe.

Eta amar preferred architecture.

Agent local project inspect korte pare, terminal command execute korte pare, files modify korte pare, Git operations perform korte pare ebong build/test results analyse korte pare. Cloud model complex reasoning, planning, coding ebong troubleshooting-er jonno use hoy.

OpenCode-er built-in tools ebong configurable permissions ei architecture-er example:

https://opencode.ai/docs/tools/

Advantages:

- Existing local project-e direct work.
- Local compiler, test runner, Git ebong installed dependencies use kora jay.
- Cloud-model-er reasoning capability benefit kora jay, local-e large LLM load na kore.
- Same agent framework-e compatible models switch kora possible.
- Specialist-agent workflow design kora jay.
- Project-er real build output ebong test failures diye results verify kora jay.
- Remote workspace-e code replicate korar extra dependency komano jay.

Disadvantages:

- AI reasoning-er jonno internet lagbe.
- Free provider quota exhaust hole model calls stop hote pare.
- Model provider-ke relevant code, prompt ba terminal output pathano hote pare.
- Agent runtime nijer-o RAM/CPU use kore.

- Permissions properly manage na korle agent unwanted changes ba dangerous commands execute korte pare.

Amar project-er jonno ei architecture best balance, karon cloud intelligence ebong local execution-er moddhe ami control rakhte pari.

D. Fully local AI agent + fully local AI model — zero internet

Agent, model inference ebong working environment shob local.

Model ebong dependencies age install kore nile, genuinely offline environment-e local files niye kaj kora possible.

Advantages:

- Model inference-er jonno third-party API quota lage na.
- Internet chara kaj kora jay, jodi task-er jonno necessary dependencies, data ebong tools age theke available thake.
- Model provider-ke inference-er jonno code pathate hoy na.
- Cloud model service down hole-o local inference cholte pare.

Disadvantages:

- Model quality local hardware ebong selected model-er capacity-r upor depend kore.
- Large models-er RAM, VRAM, storage ebong compute requirement substantial.
- Long context, complex code reasoning ebong high-concurrency workload slow hote pare.
- Model files download, update ebong maintain korte hoy.
- Local model-er nijer tool-calling, instruction following ebong coding reliability-o verify korte hoy.

Amar Acer Aspire F5-573G-er i5-7200U, 8 GB RAM ebong 4 GB GeForce 940MX configuration-er moto hardware-e small quantized model diye basic kaj kora realistic. Kintu large software project-er complex reasoning-er jonno 16+ agent-ke ekshathe capable local model diye chalano practical resource challenge hote pare.

Small local models-ke ami file search, simple transformations, summaries ebong predictable helper tasks-e use korte pari. Kintu ekhonkar hardware-e shob complex reasoning locally shift korle speed ebong quality compromise howar possibility beshi.

Architecture comparison table

Factor	Normal chatbot	Cloud agent	Local agent + cloud model	Fully local/offline
AI reasoning	Cloud	Cloud	Cloud	Local

Factor	Normal chatbot	Cloud agent	Local agent + cloud model	Fully local/offline
Main working environment	Product-dependent	Remote	Local	Local
Existing local repo access	Integration/manual work lagte pare	Sync/bridge lagte pare	Direct, with permission	Direct, with permission
Actual local build/test	Usually manual/integration	Local access bridge lagte pare	Natural fit	Natural fit
Cloud AI capability	Service-dependent	Service-dependent	Selected cloud model-er capability	Selected local model-er capability
Internet dependency	Usually yes	Yes	Model calls-er jonno yes	Proper setup-er por no
Free quota risk	Product-specific	Task/compute quota	Provider/API quota	No hosted inference quota
Local hardware demand	Low	Low	Low–moderate	Highest
Privacy	Provider policy-dependent	Remote workspace/provider-dependent	Mixed	Properly configured hole strongest locality
Amar project-er fit	Useful assistant	Remote tasks-er jonno useful	**Best overall fit**	Future optional fallback

---

4. Keno local agent + cloud model amar project-er jonno normal chatbot ba cloud-only agent-er cheye beshi suitable?

Eta sudhu preference-er bepar na. Amar actual workflow-er requirement-er sathe architecture-er fit-er bepar.

4.1 Code generate kora ebong verified code change ek jinish na

Ekta chatbot theoretically correct code dite pare, kintu code-ta project-e apply korar pore build pass korbe kina, related API break korbe kina, database schema conflict hobe kina, ba existing behavior nosto hobe kina — egulo actual verification chara confirm kora jay na.

Local agent project inspect kore change apply korte ebong available local tests execute korte pare.

Tarpor test-er actual error abar reasoning model-er kache pathano jay.

Ei feedback loop amar development speed-er jonno important.

4.2 Amar codebase-er context interconnected

Project-er frontend, backend, database, shared configuration, authentication, real-time functionality ebong background processing-er moddhe dependency ache.

Ekta isolated snippet-er answer diye shob dependency automatically bujha jay na.

Local agent relevant files search kore existing implementation inspect korte pare, dependency trace korte pare ebong targeted changes verify korte pare.

4.3 Parallel specialist-agent workflow

Amar 16+ sub-agent approach-er objective holo boro kaj-ke manageable, specialized task-e divide kora.

Udahoron:

- Repository mapping and dependency discovery.
- Frontend-specific implementation.
- Backend/API implementation.
- Database/schema investigation.
- Bug reproduction and debugging.
- Unit/integration testing.
- Build-error investigation.
- Security-sensitive logic review.
- Architecture consistency checking.
- Final integration and verification.

Shob agent-ke eki kaj deya ba prottek agent-ke maximum model diye chalano amar objective na.

Proper delegation mane duplicate work komano, ownership clear rakha, unnecessary overlap prevent kora, ebong integration-er age output verify kora.

Cloud-only agent diyeo subagents possible hote pare, platform-er upor depend kore. Kintu amar advantage holo agent-er working environment amar existing local project-er sathe connected.

4.4 Local environment-e real verification

Amar project-er dependency version, configuration, local database setup, scripts ebong build tools already local development workflow-er part.

Agent local test runner ebong terminal use korle actual observed output-er upor next decision nite pare.

Ete remote clone sync ebong local environment mismatch-er extra risk komano jay.

4.5 Model provider-er upor kom architectural lock-in

Local agent ebong cloud model alada layer.

Jodi ekta provider temporarily unavailable hoy, theoretically same compatible local agent-ke onno provider ba model-er sathe configure kora jay.

Kintu eta automatic guarantee na. Alternative model-er tool calling, context size, API compatibility, output quality ebong availability verify korte hoy.

Ei separation-er karone amar whole development architecture ekta AI model-er shathe permanently tied thake na.

---

5. Amar 16+ sub-agent strategy keno carefully planned chilo?

Ami sudhu beshi agent chalano-ke success-er definition dhorini.

Amar intended strategy holo task decomposition + role specialization + appropriate model choice + verification.

Large project-er khetre ekta task-er complexity ar ekta onno task-er complexity same na.

Ekta agent-ke file rename korte je reasoning lage, authentication architecture review korte tar cheye beshi careful reasoning dorkar.

Tai model assignment task onujayi kora uchit.

Agent/task category	Appropriate approach	Keno
File discovery, symbol search	Deterministic search tools + efficient model	Simple search-er jonno strongest model waste kora dorkar nei
Routine code modification	Balanced coding model	Everyday implementation-e capability, speed ebong quota balance dorkar
Test writing and simple fixes	Balanced model + test runner	Test results actual correctness-er evidence dey
Build error investigation	Logs + balanced model; failure-e escalate	Compiler output uncertainty komate help kore
Cross-module architecture	Strongest suitable free model	Multiple dependencies and trade-offs consider korte hoy
Complex database/auth/security changes	Strong model + independent review + tests	Failure-er impact beshi
Final integration review	Independent reviewer + deterministic checks	Same implementation agent nijer mistake miss korte pare

Ei design-er underlying principle:

Strongest model shob jaygay use kora efficiency na; suitable model diye task successful kore verify kora holo efficiency.

16+ agent-er ekta important technical limitation

16+ agent thaka mane request limit 16 bhag hoye jay na.

Ulto, agent jodi nijer reasoning, tool calls, retries, summaries ebong follow-up analysis kore, tahole total model request onek bere jete pare.

Jodi 16 ta agent ekshathe kaaj start kore, request burst, concurrency limit ebong token limit hit korar possibility thake.

Tai amar workflow-e task mapping-er sathe sathe aro kichu control important:

- Unnecessary parallel requests reduce kora.
- Duplicate codebase analysis avoid kora.
- Agent-er output choto, actionable ebong task-specific rakha.

- Model call-er age `rg` , Git, compiler, type-checker ebong test runner-er moto deterministic tools use kora.
- Fixed retry budget rakha.
- Repeated failure hole eki prompt diye endless retry na kore notun evidence collect kora.
- Critical change-er pore tests ebong review mandatory rakha.

Eta quota save korar sathe sathe correctness-o improve korte pare.

---

6. Completely free thakte chaile strongest free model shob kaj-e use korbo keno na?

Ei jaygata amar model-selection strategy-r core.

Dhora jak, duita model ache.

Model A: Beshi powerful, kintu free availability ba request quota restrictive.

Model B: Balanced intelligence, coding-er jonno sufficiently capable, ebong usable free allowance comparatively generous.

Jodi shob task Model A diye kori, tahole routine file exploration, normal component implementation, basic test writing ebong simple debugging-o scarce high-end quota consume korbe.

Erpor difficult architecture decision ba complex bug-er shomoy Model A unavailable hoye jete pare.

Onnodike, shob task Model B diye korleo difficult task-e additional retry, failed tool calls ebong incorrect changes-er risk thakbe.

Tai best strategy holo **balanced default + evidence-based escalation + verification**.

Model capability versus practical output

Ami sudhu benchmark score ba model-er marketing label diye model choose kori na.

Amar relevant factors holo:

1. **Coding reliability:** Existing conventions follow kore working changes korte pare kina.
2. **Tool-use capability:** File, terminal ebong test output-er sathe thikbhabe interact kore kina.
3. **Context handling:** Relevant codebase context dhore rakhte pare kina.
4. **Instruction following:** Task scope maintain kore, unrelated changes kom kore kina.
5. **Quota:** Free request, token, concurrency ba daily usage limit kemon.
6. **Latency:** Iterative development-er jonno response speed practical kina.
7. **Recovery capability:** Error hole evidence use kore previous mistake correct korte pare kina.

8. **Verification efficiency:** Successful result pete koto repair attempt lagche.

Ekta model beshi intelligent holeo, free quota sesh hoye gele amar workflow-er jonno oi moment-e seta unavailable.

Ekta balanced model sufficiently capable hole beshi task complete korte parle, overall output per day better hote pare.

Ekta simplified example

Dhoro, amar available allowance-e 100 ta request kora possible — eta shudhu illustrative example, kono provider-er actual current quota-r claim na.

Jodi simple kaj-e prottekbar highest-end model use kori, tahole 100 request khub taratari shesh hote pare.

Jodi deterministic tools ebong balanced model diye routine task complete kori, tahole high-end model-er requests critical task-er jonno preserve kora possible.

Tobe balanced model automatically beshi efficient hobe, eta guarantee na. Jodi se ekta task complete korte 8 ta retry ney, ar stronger model 2 ta request-e verified result dey, tahole stronger model-i oi task-er jonno better.

Tai amar goal **minimum model cost** na, abar **maximum model intelligence**-o na. Amar goal holo successful, verified work-er jonno minimum reasonable quota consumption.

---

7. Dynamic model routing: kon task-e kon model dorkar seta kivabe decide korbo?

Ami ekta static rule chai na je shob kaj ekta fixed model-i korbe.

Ami chai task complexity, uncertainty, failure evidence ebong model availability onujayi model select hok.

Tier 1 — Lightweight / low-complexity tasks

Udahoron:

- File ba symbol khuja.
- Configuration value identify kora.
- Small and straightforward edits.
- Existing pattern follow kore simple component add kora.

- Basic documentation changes.
- Formatting ebong simple tests.

Approach:

First deterministic tools use kora. Reasoning dorkar hole efficient free model use kora.

Tier 2 — Balanced everyday coding

Udahoron:

- Standard React component implementation.
- Existing API endpoint-er routine change.
- Unit tests add kora.
- Predictable TypeScript/JavaScript bug fix.
- Known pattern follow kore service add kora.

Approach:

Balanced coding model default, tarpor build, type-check, lint ebong tests diye verify.

Tier 3 — Complex reasoning

Udahoron:

- Cross-module dependency change.
- Difficult bug jekhane multiple possible root cause ache.
- Database schema migration.
- Authentication/authorization behavior.
- Race conditions, real-time state consistency.
- Significant architecture refactor.
- Security-sensitive modifications.

Approach:

Strongest suitable available free model, targeted context, clear constraints, independent review ebong actual test evidence.

Tier 4 — Failed task escalation

Jodi balanced model repeatedly fail kore, tokhon automatic retry loop endless chaliye quota waste kora uchit na.

Instead:

1. Actual error ebong changed files collect.
2. Previous failed attempts summarize.
3. Determine whether task complexity estimate vul chilo.
4. Stronger available model-e escalate.
5. Focused fix implement kore verification run.
6. Stronger model-o unavailable hole task pause kore alternative provider ba manual investigation use kora.

Router-er decision logic

Model choose korar shomoy task description alone enough na.

Relevant signals:

- Koto file/component affected hobe.
- Dependency graph koto complex.
- Root cause known naki uncertain.
- Change-er failure impact koto.
- Available tests ache kina.
- Previous attempt fail koreche kina.
- Selected model tool calling support kore kina.
- Model-er context limit enough kina.
- Free quota and concurrency available ache kina.

Task classify korar jonno alada expensive model call use korar dorkar-o nei. Onnek khetre lightweight rules, task metadata ebong previous success/failure history enough initial routing dite pare.

Dynamic routing mane prottek task-e strongest model automatically khuja na. Dynamic routing mane task-er current evidence onujayi suitable model choose kora.

---

8. OpenCode, Kilo Code, Agent Zero, ebong other options-er practical comparison

Amar original workflow-er replacement khujte hole sudhu AI model-er intelligence dekhle hobe na. Tooling, project access, free allowance, account requirements, privacy, concurrency ebong model switching-o consider korte hobe.

OpenCode

Advantages:

- Local coding-agent workflow.
- Existing repository/terminal integration.
- Multiple provider/model configuration support.
- Agent tools ebong permission control.
- Amar existing development workflow-er sathe already integrated.

Current risk:

OpenCode-er built-in free-model service-e server-side usage limits thakte pare. Current GitHub reports dekhiyeche je displayed reset-er poreo kichu user limit error report korechen.

Conclusion: Amar architecture-er jonno OpenCode fit chilo; kintu ekta free hosted-model gateway-ke only dependency banano unnecessary operational risk.

Official docs: https://opencode.ai/docs/

Kilo Code

Kilo Code-er official documentation onujayi `kilo-auto/free` available free models-er moddhe automatically route kore. Documentation-e free-model gateway-r published limit 200 requests per hour per IP deya ache.

Sources:

- https://kilo.ai/docs/getting-started/using-kilo-for-free
- https://kilo.ai/docs/getting-started/rate-limits-and-costs

Kintu 200 requests/hour mane guaranteed 200 successful complex coding tasks na.

Individual upstream providers-er restrictions thakte pare; model availability change hote pare; background tasks-o usage consume korte pare. Kilo-r Auto Free data-handling terms-o evaluate korte hobe, karon kichu model provider prompts ebong outputs log korte ba service improvement-er jonno use korte pare.

Conclusion: Free-model fallback-er jonno Kilo Code worth evaluating, kintu amar current workflow replace korar age same repository-te agent behavior, subagent support, model choices ebong successful task-per-request test korte hobe.

Agent Zero

Agent Zero general-purpose agent framework hisebe browser, terminal, tools ebong multiple agents-er broader workflow support kore. Official documentation-e local files access-er jonno A0 CLI connector ebong Docker-based environment describe kora ache.

Sources:

- https://www.agent-zero.ai/p/docs/
- https://github.com/agent0ai/agent-zero

Kintu Agent Zero install korlei AI inference unlimited free hoy na. Free model provider ba API gateway-er allowance ekhaneo applicable hobe. Dockerized environment ebong host bridge-er setup-o manage korte hobe.

Conclusion: Broader computer/browser automation dorkar hole Agent Zero strong candidate. Amar immediate priority jodi existing local coding workflow ebong sustainable free inference hoy, tahole OpenCode-er alternative model-routing strategy first test kora beshi direct.

OpenRouter

OpenRouter-er official pricing page-e free tier-er jonno 50 requests per day ebong 20 requests per minute published ache. Qualifying $10 credit purchase korar por free-model daily allowance barar policy-o documentation-e ache, kintu seta completely no-payment strategy-r sathe compatible na.

Source: https://openrouter.ai/pricing

Conclusion: Model variety ebong provider routing-er jonno useful, kintu free-only workflow-te published free request limit carefully calculate korte hobe.

GroqCloud

Groq model-specific free API limits publish kore, jar moddhe RPM, RPD, TPM ebong TPD thakte pare. Exact allowance model ebong account-er upor depend kore.

Official documentation: https://console.groq.com/docs/rate-limits

Conclusion: Alternative provider hisebe useful hote pare, kintu account/API key setup, available model, tool-use reliability ebong current free quota verify kore tarpor workflow-e integrate kora uchit.

Comparison

Option	Local coding environment	Free model strategy	Main concern	Amar evaluation
OpenCode	Strong	Built-in free service or configured providers	Free service availability/reset	Primary workflow-er sathe already fit

Option	Local coding environment	Free model strategy	Main concern	Amar evaluation
Kilo Code	Local coding agent	Auto Free + individual free models	Gateway/upstream limits and data policy	Important fallback candidate
Agent Zero	Broader local agent environment	Configured cloud or local models	Docker/resources and provider quota	Browser/computer-level task-er jonno
OpenRouter	Agent alada; API provider	Free model catalogue	Free daily request limits	Flexible provider option
GroqCloud	Agent alada; API provider	Model-specific free allowance	Token/request limits and API setup	Alternative cloud inference
Fully local model	Fully local	No hosted inference quota	Local hardware/model capability	Offline fallback, not primary large-project engine

Ei comparison-er objective kono ekta brand-ke universally best bola na. Amar actual environment-e kon combination highest verified output, reliable model access ebong lowest operating cost dey seta determine kora.

---

9. Amar proposed long-term architecture

Ami je design-ta prefer kori seta holo:

Local development agent + task-aware model routing + multiple fallback providers + deterministic verification.

Layer 1: Local project execution

Agent local repository-te kaj korbe. Files, terminal, Git, build, lint, test ebong development tools project environment-e execute hobe.

Layer 2: Specialist task orchestration

Task-ke clearly defined unit-e divide kora hobe. Prottek agent-er responsibility, permitted files, expected output ebong verification criteria thakbe.

Shob task-e 16+ agent simultaneously start korar bodole task dependency ebong available quota onujayi concurrency manage kora uchit.

Layer 3: Model selection

Routine task-er jonno balanced model.

Complex reasoning-er jonno strongest suitable free model.

Simple search, formatting, static analysis ebong repeatable checks-er jonno deterministic tools.

Layer 4: Provider fallback

Primary free model unavailable hole compatible alternate model/provider try kora jabe.

Kintu model switching korlei independent fresh quota guaranteed na. Different providers-er allowance alada hote pare; same gateway-r free models shared restriction-er moddheo porte pare. Eta current provider-er actual documented policy ebong observed behavior diye verify korte hobe.

Layer 5: Usage control

Track kora uchit:

- Model/provider.
- Request count ebong available quota signals.
- Failure type.
- Retry count.
- Input/output token consumption, jodi available hoy.
- Task success ebong verification result.
- Escalation keno hoyeche.
- Kon provider/model-e task successful hoyeche.

Layer 6: Verification and safety

Code changes-er pore relevant tests, build, type-check, lint ebong security review run kora uchit.

Destructive shell command, database migration, authentication, credentials ebong production-related changes human approval chara unrestricted bhabe execute kora uchit na.

Eta important, karon stronger model holeo mistake possible.

---

10. Amar immediate problem-er practical interpretation

Amar concern holo OpenCode sudhu ekta message-er error dicche na; amar planned development workflow-er reliability affected hocche.

Jodi shob agent ekta hosted free gateway-r upor dependent hoy, tahole gateway limit/reset issue puro pipeline-ke block korte pare.

Tai ami ei problem-ke duita independent issue hisebe dekhi:

Issue A — OpenCode free-tier reliability

Current public reports amar symptom-er sathe match kore. Current main discussion-e v2.1.4 user-der reset-er poreo free model access fail korar report ache.

Latest version, exact error, OS, model ID, reset time ebong new-session behavior document kora uchit. OpenCode team-er nijer guidance onujayi current v2 use kora ebong exact symptom main free-tier issue-te report kora useful.

Main thread: https://github.com/anomalyco/opencode/issues/59387

Issue B — Amar project's inference architecture-er resilience

Jodio OpenCode-er current problem pore resolve hoy, amar workflow-ke ekta hosted free service-er availability-r upor completely dependent rakha ideal na.

Amar long-term solution holo compatible provider/model options maintain kora, task-based routing use kora, request fan-out control kora ebong deterministic verification-er upor beshi depend kora.

Ete ekta provider unavailable hole shob local development work permanently blocked hoye thakar risk komano possible.

---

11. Amar final reasoning

Ami normal AI chatbot reject korchi na, ebong cloud-based agents-o useless bolchi na.

Ami bolchi, amar specific project, hardware, development style ebong $0 budget-er jonno ora automatically best replacement na.

Ami chai:

- AI amar actual local project niye kaj koruk.
- Multiple specialized tasks efficiently coordinate hok.
- Available free models intelligently use hok.
- Routine kaj-e unnecessarily scarce strongest model consume na hok.
- Complex task-e stronger reasoning preserve kora hok.
- Actual build/test output diye code verify hok.

- Model ba provider unavailable hole alternative path thakuk.
- Paid API usage accidentally start na hok.
- My local development environment ebong project state-er upor amar control thakuk.

Ei requirement-gulor combination-er jonno **local agent + cloud model architecture** amar kache shobcheye practical.

Ar free-only setup-e ami balanced coding model-ke default hisebe choose korbo, karon prottek task-er jonno maximum intelligence dorkar hoy na. Tarpor task complexity, failure evidence ebong verification result onujayi stronger model-e escalate korbo.

Model-er highest benchmark score, model picker-e beshi options, ba published request limit — egulor konotai ekla successful project delivery guarantee kore na.

Amar real measure holo:

Koto efficiently ami amar actual codebase-e correct, tested, maintainable change deliver korte parchi — free resources use kore.

Tai amar solution-er focus holo sudhu "good AI use koro" na. Focus holo suitable agent architecture, intelligent task decomposition, sustainable free-model routing, controlled parallelism, ebong evidence-based verification.

Ei approach-er maddhome ami local development environment-er control maintain kore cloud model-er reasoning benefit nite pari, abar ekta free provider-er quota issue-ke amar entire development strategy-r permanent bottleneck hote dei na.

Final decision: Local coding agent-ke primary workflow hisebe maintain kora, balanced free coding models diye daily development chalano, difficult task-e strongest suitable free model use kora, ebong OpenCode free-tier issue persist korle independently tested alternative provider/agent route maintain kora — etai amar project-er jonno shobcheye reasoned path.